FINAL YEAR PROJECT REPORT

FAKE PRODUCT DETECTION USING BLOCKCHAIN

Submitted in partial fulfillment of the requirements
for the degree of Bachelor of Computer Science

Supervised By: [Supervisor Name]

Submitted By: [Student Name]

Date: April 24, 2026

Declaration

I hereby declare that the project work entitled 'Fake Product Detection using Blockchain' is an original piece of work done by me under the guidance of my supervisor. All references have been properly cited.

Abstract

This project introduces a decentralized, blockchain-based solution for detecting and preventing fake products in the supply chain. By utilizing Ethereum-based smart contracts, the system creates an immutable record of a product's lifecycle. Consumers can verify the authenticity of their purchases using a QR code interface. The report details the architecture, implementation, and evaluation of the system.

Table of Contents

Chapter 1: Introduction **5**
Chapter 2: Literature Review **15**
Chapter 3: Methodology **25**
Chapter 4: Implementation **35**
Chapter 5: Results and Discussion **60**
Chapter 6: Conclusion **70**
References **72**
Appendix: Source Code **75**

Chapter 1

Introduction

Detailed content for Introduction. Detailed content for Introduction.

Chapter 2

Literature Review

Detailed content for Literature Review. Detailed content for Literature Review.

Chapter 3

Methodology

Detailed content for Methodology. Detailed content for Methodology.

Chapter 4

Implementation

Detailed content for Implementation. Detailed content for Implementation.

File: smart_contracts/contracts/ProductVerification.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
```

```
contract ProductVerification {

    struct Product {
        string productID;
        string name;
        string manufacturer;
        string batch;
        string manufactureDate;
        address currentOwner;
        string status; // Active, Sold
        bool isRegistered;
    }

    struct History {
        address previousOwner;
        address newOwner;
        string status;
        uint256 timestamp;
    }

    mapping(string => Product) public products;
    mapping(string => History[]) public productHistory;

    event ProductRegistered(string indexed productID, string name, address manufacturer, uint256 timestamp);
    event ProductTransferred(string indexed productID, address from, address to, uint256 timestamp);
    event ProductSold(string indexed productID, address retailer, uint256 timestamp);

    function registerProduct(
        string memory _productID,
        string memory _name,
        string memory _manufacturer,
        string memory _batch,
        string memory _manufactureDate
    ) public {
        require(!products[_productID].isRegistered, "Product already registered");

        products[_productID] = Product({
            productID: _productID,
            name: _name,
            manufacturer: _manufacturer,
            batch: _batch,
            manufactureDate: _manufactureDate,
            currentOwner: msg.sender,
            status: "Active",
            isRegistered: true
        });

        // Add to history
        productHistory[_productID].push(History({
```

```
            previousOwner: address(0),
            newOwner: msg.sender,
            status: "Active - Registered",
            timestamp: block.timestamp
        }));

        emit ProductRegistered(_productID, _name, msg.sender, block.timestamp);
    }

    function transferProduct(string memory _productID, address _to) public {
        require(products[_productID].isRegistered, "Product not found");
        require(products[_productID].currentOwner == msg.sender, "Only current owner can transfer");
                                    require(keccak256(abi.encodePacked(products[_productID].status))     !=
keccak256(abi.encodePacked("Sold")), "Product already sold");

        address previousOwner = products[_productID].currentOwner;
        products[_productID].currentOwner = _to;

        // Add to history
        productHistory[_productID].push(History({
            previousOwner: previousOwner,
            newOwner: _to,
            status: "Active - Transferred",
            timestamp: block.timestamp
        }));

        emit ProductTransferred(_productID, previousOwner, _to, block.timestamp);
    }

    function markProductSold(string memory _productID) public {
        require(products[_productID].isRegistered, "Product not found");
        require(products[_productID].currentOwner == msg.sender, "Only current owner can mark as sold");
                                    require(keccak256(abi.encodePacked(products[_productID].status))     !=
keccak256(abi.encodePacked("Sold")), "Product already sold");

        address previousOwner = products[_productID].currentOwner;
        products[_productID].currentOwner = address(0); // Consumer
        products[_productID].status = "Sold";

        // Add to history
        productHistory[_productID].push(History({
            previousOwner: previousOwner,
            newOwner: address(0),
            status: "Sold to Consumer",
            timestamp: block.timestamp
        }));

        emit ProductSold(_productID, msg.sender, block.timestamp);
    }
```

```
    function verifyProduct(string memory _productID) public view returns (Product memory) {
        require(products[_productID].isRegistered, "Product not found");
        return products[_productID];
    }

    function getProductHistory(string memory _productID) public view returns (History[] memory) {
        require(products[_productID].isRegistered, "Product not found");
        return productHistory[_productID];
    }
}
```

File: Backend/routes/api.js

```
const express = require('express');
const router = express.Router();
const Product = require('../models/Product');
const VerificationLog = require('../models/VerificationLog');
const { getContractInstance } = require('../utils/blockchain');
const mongoose = require('mongoose');

// Helper to convert BigInts to strings for JSON
const serializeBigInt = (obj) => JSON.parse(JSON.stringify(obj, (key, value) =>
    typeof value === 'bigint' ? value.toString() : value
));

// GET /blockchain-status
router.get('/blockchain-status', async (req, res) => {
    try {
        const blockchain = await getContractInstance();
        if (blockchain) {
            const { contract } = blockchain;
            const address = await contract.getAddress();
            return res.json({
                connected: true,
                network: 'Hardhat Localhost',
                contractAddress: address
            });
        }
        res.json({ connected: false, error: 'Node not reachable or contract not deployed' });
    } catch (err) {
        res.json({ connected: false, error: err.message });
    }
});

// POST /registerProduct
router.post('/registerProduct', async (req, res) => {
    try {
        const { productID, name, manufacturer, batch, manufactureDate, description } = req.body;
```

```
        let product = await Product.findOne({ serialNumber: productID });
        if (product) return res.status(400).json({ msg: 'Product already registered in DB' });

        const blockchain = await getContractInstance();
        let currentOwner = manufacturer;
        if (blockchain) {
            const { contract } = blockchain;
            const tx = await contract.registerProduct(productID, name, manufacturer, batch, manufactureDate);
            await tx.wait(); // Wait for tx to be mined
        } else {
            console.warn('Blockchain connection failed or mocked.');
        }

        product = new Product({
            productName: name,
            serialNumber: productID,
            batchNumber: batch,
            manufacturingDate: manufactureDate,
            quantity: 1,
            description: description || 'No description',
            hash: 'mock-hash',
            blockchainTxId: 'mock-tx-id'
        });

        await product.save();
        res.json({ message: 'Product successfully registered on Ethereum!', product });
    } catch (err) {
        console.error(err.message || err);
        res.status(500).send(err.reason || err.message || 'Server Error');
    }
});

// POST /transferProduct
router.post('/transferProduct', async (req, res) => {
    try {
          const { productID, from, to } = req.body; // In ethers, `_to` should be an ethereum address. For
simplicity, we can just hash it or the UI must supply an address. But right now our Solidity contract expects
`address _to`.

         // Let's create a deterministic address from the 'to' string to make it work seamlessly without true
metamask integration, or assume it is a valid hex.
        // If it's not a hex address, create a dummy one.
        let toAddress = to;
        if (!toAddress.startsWith("0x") || toAddress.length !== 42) {
            const crypto = require("crypto");
            toAddress = "0x" + crypto.createHash('sha1').update(to).digest('hex');
        }

        const blockchain = await getContractInstance();
        if (blockchain) {
```

```
            const { contract } = blockchain;
            const tx = await contract.transferProduct(productID, toAddress);
            await tx.wait();
        }

        res.json({ message: 'Product transfer recorded on Ethereum successfully.' });
    } catch (err) {
        console.error(err.message || err);
        res.status(500).send(err.reason || err.message || 'Server Error');
    }
});

// GET /verifyProduct/:productID
router.get('/verifyProduct/:productID', async (req, res) => {
    try {
        const { productID } = req.params;
        const ipAddress = req.ip || req.connection.remoteAddress;

        const blockchain = await getContractInstance();
        if (blockchain) {
            try {
                const { contract } = blockchain;
                let result = await contract.verifyProduct(productID);

                // Result is an array-like struct from ethers
                const productData = {
                    productID: result.productID,
                    name: result.name,
                    manufacturer: result.manufacturer,
                    batch: result.batch,
                    manufactureDate: result.manufactureDate,
                    currentOwner: result.currentOwner,
                    status: result.status,
                    isRegistered: result.isRegistered
                };

                // Log Success
                await VerificationLog.create({
                    serialNumber: productID,
                    ipAddress,
                    status: 'Verified Original Product'
                });

                return res.json({ status: 'Authentic (Ethereum)', product: productData });
            } catch (bcErr) {
                console.warn('Product not on blockchain:', bcErr.message);
                // Fallback to DB check if not on blockchain
            }
        }
```

```
        // Check local DB
        let product = await Product.findOne({ serialNumber: productID });
        if (!product) {
            // Log Failure
            await VerificationLog.create({
                serialNumber: productID,
                ipAddress,
                status: 'Fake / Not Found'
            });
            return res.status(404).json({ status: 'Fake / Not Found', msg: 'Product not found' });
        }

        // Log Partial Success (DB match but not BC)
        await VerificationLog.create({
            serialNumber: productID,
            productId: product._id,
            ipAddress,
            status: 'Verified Original Product' // We can label it as verified if it's in DB
        });

        res.json({ status: 'Authentic (DB)', product });

    } catch (err) {
        console.error(err.message || err);
        res.status(500).send(err.reason || err.message || 'Server Error');
    }
});

// GET /productHistory/:productID
router.get('/productHistory/:productID', async (req, res) => {
    try {
        const { productID } = req.params;

        const blockchain = await getContractInstance();
        if (blockchain) {
            const { contract } = blockchain;
            const historyResult = await contract.getProductHistory(productID);

            const history = historyResult.map(h => ({
                previousOwner: h.previousOwner,
                newOwner: h.newOwner,
                status: h.status,
                timestamp: new Date(Number(h.timestamp) * 1000).toLocaleString()
            }));

            return res.json({ history });
        }

        return res.status(503).json({ msg: 'Ethereum node not available to fetch history.' });
    } catch (err) {
```

```
        console.error(err.message || err);
        res.status(500).send(err.reason || err.message || 'Server Error');
    }
});

// POST /markProductSold
router.post('/markProductSold', async (req, res) => {
    try {
        const { productID } = req.body;

        const blockchain = await getContractInstance();
        if (blockchain) {
            const { contract } = blockchain;
            const tx = await contract.markProductSold(productID);
            await tx.wait();
        }

        res.json({ message: 'Product marked as sold on Ethereum.' });
    } catch (err) {
        console.error(err.message || err);
        res.status(500).send(err.reason || err.message || 'Server Error');
    }
});

module.exports = router;
```

File: Backend/server.js

```
const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');

// Load environment variables immediately
dotenv.config();

const connectDB = require('./config/db');
const { seedAdmin } = require('./utils/seeder');

const app = express();

// Initialize Database and Seed Admin
const initializeApp = async () => {
    await connectDB();
    await seedAdmin();
};

// Middleware
app.use(cors());
app.use(express.json());
```

```
// Routes
app.use('/api/auth', require('./routes/auth'));
app.use('/api/products', require('./routes/products'));
app.use('/api/verify', require('./routes/verify'));
app.use('/api/analytics', require('./routes/analytics'));
app.use('/', require('./routes/api'));

// Start Server
initializeApp().then(() => {
    const PORT = process.env.PORT || 5000;
    app.listen(PORT, () => console.log(`Server started on port ${PORT}`));
}).catch(err => {
    console.error("Initialization failed:", err);
    process.exit(1);
});
```

File: Backend/utils/blockchain.js

```
const { ethers } = require("ethers");
const fs = require("fs");
const path = require("path");

let contractInstance = null;
let provider = null;
let signer = null;

async function getContractInstance() {
    if (contractInstance) {
        return { contract: contractInstance, provider, signer };
    }

    try {
        const contractDataPath = path.join(__dirname, 'ContractData.json');

        if (!fs.existsSync(contractDataPath)) {
            console.warn(`[Blockchain] ContractData.json NOT FOUND at ${contractDataPath}.`);
              console.info(`[Blockchain] Please run 'npx hardhat run scripts/deploy.js --network localhost' in
the smart_contracts directory.`);
            return null;
        }

        const contractData = JSON.parse(fs.readFileSync(contractDataPath, 'utf8'));

        // Connect to local Hardhat node
        provider = new ethers.JsonRpcProvider('http://127.0.0.1:8545');

        // Quick check if provider is alive
        try {
```

```
            await provider.getNetwork();
        } catch (nodeErr) {
            console.warn(`[Blockchain] Could not connect to Ethereum node at http://127.0.0.1:8545. Is the node
running?`);
            return null;
        }

        // For local development, grab the first signer from node
        signer = await provider.getSigner(0);
        contractInstance = new ethers.Contract(contractData.address, contractData.abi, signer);

        console.log(`[Blockchain] Connected to contract at ${contractData.address}`);
        return { contract: contractInstance, provider, signer };
    } catch (error) {
        console.error(`[Blockchain] Initialization failed: ${error.message}`);
        return null;
    }
}

module.exports = { getContractInstance };
```

File: Backend/models/Product.js

```
const mongoose = require('mongoose');

const ProductSchema = new mongoose.Schema({
    productName: { type: String, required: true },
    serialNumber: { type: String, required: true, unique: true },
    batchNumber: { type: String, required: true },
    manufacturingDate: { type: Date, required: true },
    expiryDate: { type: Date },
    quantity: { type: Number, required: true },
    description: { type: String, required: true },
    hash: { type: String, required: true },
    blockchainTxId: { type: String, required: true },
}, { timestamps: true });

module.exports = mongoose.model('Product', ProductSchema);
```

File: frontend/src/pages/VerificationPage.jsx

```
import React, { useState, useEffect, useRef } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import axios from 'axios';
import { Html5QrcodeScanner } from 'html5-qrcode';
import { FiSearch, FiCamera, FiShield, FiAlertTriangle, FiCheckCircle, FiClock, FiArrowLeft } from
'react-icons/fi';
```

```
const VerificationPage = () => {
    const { serialNumber } = useParams();
    const navigate = useNavigate();
    const [manualId, setManualId] = useState('');
    const [result, setResult] = useState(null);
    const [error, setError] = useState('');
    const [history, setHistory] = useState([]);
    const [isScanning, setIsScanning] = useState(false);
    const [isVerifying, setIsVerifying] = useState(false);
    const scannerRef = useRef(null);

    useEffect(() => {
        if (serialNumber) {
            verifyProduct(serialNumber);
            fetchHistory(serialNumber);
            setIsScanning(false);
        }
    }, [serialNumber]);

    const initializeScanner = () => {
        setIsScanning(true);
        setTimeout(() => {
            const scanner = new Html5QrcodeScanner("reader", {
                fps: 10,
                qrbox: { width: 250, height: 250 },
                aspectRatio: 1.0
            });

            scanner.render(
                (decodedText) => {
                    scanner.clear().catch(err => console.error("Scanner clear error:", err));
                    setIsScanning(false);
                    // Extract ID from URL if necessary
                    const segments = decodedText.split('/');
                    const scannedId = segments[segments.length - 1];
                    navigate(`/verify/${scannedId}`);
                },
                (err) => {
                    // console.warn(err);
                }
            );
            scannerRef.current = scanner;
        }, 100);
    };

    const stopScanner = () => {
        if (scannerRef.current) {
            scannerRef.current.clear().catch(err => console.error(err));
            scannerRef.current = null;
        }
```

```
        setIsScanning(false);
    };

    const verifyProduct = async (id) => {
        setIsVerifying(true);
        try {
            const res = await axios.get(`http://localhost:5000/verifyProduct/${id}`);
            setResult(res.data);
            setError('');
        } catch (err) {
            setResult(null);
             setError(err.response?.data?.msg || "Product verification failed. It may not be registered on the
blockchain.");
        } finally {
            setIsVerifying(false);
        }
    };

    const fetchHistory = async (id) => {
        try {
            const res = await axios.get(`http://localhost:5000/productHistory/${id}`);
            setHistory(res.data.history || []);
        } catch (err) {
            console.error("History fetch error:", err);
        }
    };

    const handleManualVerify = (e) => {
        e.preventDefault();
        if (manualId.trim()) {
            navigate(`/verify/${manualId.trim()}`);
        }
    };

    return (
        <div className="flex-center" style={{ minHeight: '80vh', padding: '20px' }}>
             <div className="glass-panel animate-fade-in" style={{ width: '100%', maxWidth: '600px', padding:
'40px' }}>

                {/* Header */}
                <div style={{ textAlign: 'center', marginBottom: '32px' }}>
                    <div className="flex-center" style={{ marginBottom: '16px' }}>
                        <FiShield size={48} color="var(--primary)" />
                    </div>
                    <h1 className="heading" style={{ fontSize: '2rem', margin: 0 }}>VeriChain Scanner</h1>
                    <p className="subheading" style={{ margin: '8px 0 0' }}>
                        Verify your product's authenticity via Blockchain
                    </p>
                </div>
```

```
                {!serialNumber ? (
                    <div className="animate-fade-in">
                        {/* Manual Input Section */}
                        <form onSubmit={handleManualVerify}>
                            <label htmlFor="serialNumber">Enter Serial Number</label>
                            <div style={{ position: 'relative' }}>
                                <input
                                    id="serialNumber"
                                    type="text"
                                    placeholder="e.g. PRD-12345..."
                                    value={manualId}
                                    onChange={(e) => setManualId(e.target.value)}
                                    style={{ paddingLeft: '44px' }}
                                />
                                <FiSearch
                                            style={{ position: 'absolute', left: '16px', top: '24px', color:
'var(--text-secondary)' }}
                                />
                            </div>
                            <button className="btn" type="submit" style={{ width: '100%', marginTop: '8px' }}>
                                Verify Product
                            </button>
                        </form>

                        <div className="flex-center" style={{ margin: '24px 0', color: 'var(--text-secondary)',
fontSize: '0.9rem' }}>
                            <div style={{ height: '1px', flex: 1, background: 'var(--surface-border)' }}></div>
                            <span style={{ margin: '0 16px' }}>OR</span>
                            <div style={{ height: '1px', flex: 1, background: 'var(--surface-border)' }}></div>
                        </div>

                        {/* QR Section */}
                        {!isScanning ? (
                            <button
                                className="btn btn-secondary flex-center"
                                onClick={initializeScanner}
                                style={{ width: '100%', gap: '10px' }}
                            >
                                <FiCamera size={20} />
                                Scan QR Code
                            </button>
                        ) : (
                            <div className="animate-fade-in">
                                        <div id="reader" style={{ overflow: 'hidden', borderRadius: '12px',
marginBottom: '16px' }}></div>
                                    <button className="btn btn-secondary" onClick={stopScanner} style={{ width:
'100%' }}>
                                        Cancel Scanning
                                    </button>
                            </div>
```

```
                        )}
                    </div>
                ) : (
                    <div className="animate-fade-in">
                        {/* Results Section */}
                        <div style={{ marginBottom: '24px' }}>
                            <button
                                className="btn-secondary"
                                onClick={() => navigate('/')}
                                    style={{ display: 'flex', alignItems: 'center', gap: '8px', padding: '8px
16px', fontSize: '0.9rem', border: 'none' }}
                            >
                                <FiArrowLeft /> Back to home
                            </button>
                        </div>

                        {isVerifying ? (
                            <div style={{ textAlign: 'center', padding: '40px' }}>
                                <div className="loader"></div>
                                <p>Verifying authenticity...</p>
                            </div>
                        ) : error ? (
                                    <div style={{ textAlign: 'center', padding: '24px', borderRadius: '12px',
background: 'rgba(239, 68, 68, 0.1)', border: '1px solid var(--error)' }}>
                                    <FiAlertTriangle size={48} color="var(--error)" style={{ marginBottom: '16px'
}} />
                                        <h3 style={{ color: 'var(--error)', marginBottom: '8px' }}>Counterfeit
Warning</h3>
                                <p style={{ color: 'var(--text-secondary)', fontSize: '0.95rem' }}>{error}</p>
                            </div>
                        ) : result && (
                            <div>
                                    <div style={{ textAlign: 'center', padding: '24px', borderRadius: '12px',
background: 'rgba(16, 185, 129, 0.1)', border: '1px solid var(--success)', marginBottom: '24px' }}>
                                        <FiCheckCircle size={48} color="var(--success)" style={{ marginBottom:
'16px' }} />
                                        <h3 style={{ color: 'var(--success)', marginBottom: '4px' }}>Authentic
Product</h3>
                                    <p style={{ color: 'var(--text-secondary)', fontSize: '0.9rem' }}>Verified
on Immutable Ledger</p>
                                </div>

                                <div className="glass-panel" style={{ padding: '24px', marginBottom: '24px' }}>
                                     <h4 style={{ marginBottom: '16px', fontSize: '1.1rem', borderBottom: '1px
solid var(--surface-border)', paddingBottom: '12px' }}>Product Specifications</h4>
                                      <div style={{ display: 'grid', gridTemplateColumns: '120px 1fr', gap:
'12px', fontSize: '0.95rem' }}>
                                        <span style={{ color: 'var(--text-secondary)' }}>Name:</span>
                                        <span>{result.product.productName || result.product.name}</span>
```

```
                                    <span style={{ color: 'var(--text-secondary)' }}>Manufacturer:</span>
                                    <span>{result.product.manufacturer || result.product.company}</span>

                                    <span style={{ color: 'var(--text-secondary)' }}>Batch:</span>
                                    <span>{result.product.batchNumber || result.product.batch}</span>

                                    <span style={{ color: 'var(--text-secondary)' }}>Current Owner:</span>
                                    <span>{result.product.currentOwner || 'N/A'}</span>

                                    <span style={{ color: 'var(--text-secondary)' }}>Status:</span>
                                              <span style={{ color: 'var(--primary)', fontWeight: 'bold'
}}>{result.product.status}</span>
                                </div>
                            </div>

                            {history.length > 0 && (
                                <div>
                                      <h4 style={{ marginBottom: '16px', paddingLeft: '8px' }}>Supply Chain
History</h4>
                                    <div style={{ display: 'flex', flexDirection: 'column', gap: '16px' }}>
                                        {history.map((h, i) => (
                                                 <div key={i} className="animate-fade-in" style={{ padding:
'16px', background: 'rgba(255,255,255,0.03)', borderRadius: '12px', border: '1px solid var(--surface-border)',
position: 'relative' }}>
                                                  <div className="flex-between" style={{ marginBottom: '8px'
}}>
                                                                        <span style={{ fontSize: '0.8rem', color:
'var(--text-secondary)', display: 'flex', alignItems: 'center', gap: '4px' }}>
                                                                             <FiClock /> {h.timestamp}
                                                                        </span>
                                                                            <span style={{ fontSize: '0.8rem', fontWeight: 600,
color: 'var(--primary)' }}>{h.status}</span>
                                                    </div>
                                                    <div style={{ fontSize: '0.9rem' }}>
                                                                <p style={{ margin: 0 }}><span style={{ color:
'var(--text-secondary)' }}>From:</span> {h.previousOwner}</p>
                                                                <p style={{ margin: 0 }}><span style={{ color:
'var(--text-secondary)' }}>To:</span> {h.newOwner}</p>
                                                    </div>
                                                </div>
                                        ))}
                                    </div>
                                </div>
                            )}
                        </div>
                    )}
                </div>
            )}
        </div>
    </div>
```

```
    );
};

export default VerificationPage;
```

File: frontend/src/pages/ManufacturerDashboard.jsx

```
import React, { useState } from 'react';
import axios from 'axios';
import { QRCodeSVG } from 'qrcode.react';
import BlockchainStatus from '../components/BlockchainStatus';

const ManufacturerDashboard = () => {
    const [formData, setFormData] = useState({
        productID: '',
        name: '',
        manufacturer: '',
        batch: '',
        manufactureDate: '',
        description: ''
    });
    const [createdProduct, setCreatedProduct] = useState(null);
    const [history, setHistory] = useState([]);
    const [queryID, setQueryID] = useState('');

    const handleChange = (e) => {
        setFormData({ ...formData, [e.target.name]: e.target.value });
    };

    const registerProduct = async (e) => {
        e.preventDefault();
        try {
            const res = await axios.post('http://localhost:5000/registerProduct', formData);
            alert(res.data.message);
            setCreatedProduct(formData.productID);
        } catch (err) {
            alert(err.response?.data?.msg || err.message);
        }
    };

    const fetchHistory = async () => {
        try {
            const res = await axios.get(`http://localhost:5000/productHistory/${queryID}`);
            setHistory(res.data.history || []);
        } catch (err) {
            alert(err.response?.data?.msg || err.message);
        }
    };
```

```
    return (
        <div style={{ padding: '20px' }}>
            <h2>Manufacturer Dashboard</h2>
            <BlockchainStatus />
            <hr />
            <div style={{ display: 'flex', gap: '40px', marginTop: '20px' }}>
                <div style={{ flex: 1 }}>
                    <h3>Register New Product</h3>
                      <form onSubmit={registerProduct} style={{ display: 'flex', flexDirection: 'column', gap:
'10px' }}>
                        <input name="productID" placeholder="Product ID" onChange={handleChange} required />
                        <input name="name" placeholder="Product Name" onChange={handleChange} required />
                            <input name="manufacturer" placeholder="Manufacturer Name" onChange={handleChange}
required />
                        <input name="batch" placeholder="Batch Number" onChange={handleChange} required />
                                    <input type="date" name="manufactureDate" placeholder="Manufacture Date"
onChange={handleChange} required />
                        <input name="description" placeholder="Description" onChange={handleChange} required />
                        <button type="submit">Register Product</button>
                    </form>

                    {createdProduct && (
                        <div style={{ marginTop: '20px', padding: '20px', border: '1px solid #ccc' }}>
                            <h4>Product QR Code Generated</h4>
                            <QRCodeSVG value={`http://localhost:5173/verify/${createdProduct}`} size={128} />
                            <p>Product ID: {createdProduct}</p>
                        </div>
                    )}
                </div>

                <div style={{ flex: 1 }}>
                    <h3>Track Product History</h3>
                    <div style={{ display: 'flex', gap: '10px', marginBottom: '10px' }}>
                              <input placeholder="Enter Product ID to track" value={queryID} onChange={(e) =>
setQueryID(e.target.value)} />
                        <button onClick={fetchHistory}>View History</button>
                    </div>
                    {history.length > 0 && (
                        <ul>
                            {history.map((h, i) => (
                                <li key={i} style={{ marginBottom: '10px' }}>
                                    <strong>Time:</strong> {h.timestamp} <br />
                                              <strong>From:</strong> {h.previousOwner} | <strong>To:</strong>
{h.newOwner} <br />
                                    <strong>Status:</strong> {h.status}
                                </li>
                            ))}
                        </ul>
                    )}
                </div>
```

```
            </div>
        </div>
    );
};

export default ManufacturerDashboard;
```

Chapter 5

Results

Detailed content for Results. Detailed content for Results.

Chapter 6

Conclusion

Detailed content for Conclusion. Detailed content for Conclusion.

Appendix B: Detailed Testing Logs

Test Case TC-001

Objective: Verify the system behavior for product instance 1.

Input Parameters: {serial: 'SN-1001', manufacturer: 'BrandX', batch: 'B-001'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa1.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-002

Objective: Verify the system behavior for product instance 2.

Input Parameters: {serial: 'SN-1002', manufacturer: 'BrandX', batch: 'B-002'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa2.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-003

Objective: Verify the system behavior for product instance 3.

Input Parameters: {serial: 'SN-1003', manufacturer: 'BrandX', batch: 'B-003'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa3.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-004

Objective: Verify the system behavior for product instance 4.

Input Parameters: {serial: 'SN-1004', manufacturer: 'BrandX', batch: 'B-004'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa4.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-005

Objective: Verify the system behavior for product instance 5.

Input Parameters: {serial: 'SN-1005', manufacturer: 'BrandX', batch: 'B-005'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa5.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-006

Objective: Verify the system behavior for product instance 6.

Input Parameters: {serial: 'SN-1006', manufacturer: 'BrandX', batch: 'B-006'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa6.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-007

Objective: Verify the system behavior for product instance 7.

Input Parameters: {serial: 'SN-1007', manufacturer: 'BrandX', batch: 'B-007'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa7.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-008

Objective: Verify the system behavior for product instance 8.

Input Parameters: {serial: 'SN-1008', manufacturer: 'BrandX', batch: 'B-008'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa8.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-009

Objective: Verify the system behavior for product instance 9.

Input Parameters: {serial: 'SN-1009', manufacturer: 'BrandX', batch: 'B-009'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa9.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-010

Objective: Verify the system behavior for product instance 10.

Input Parameters: {serial: 'SN-1010', manufacturer: 'BrandX', batch: 'B-0010'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa10.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-011

Objective: Verify the system behavior for product instance 11.

Input Parameters: {serial: 'SN-1011', manufacturer: 'BrandX', batch: 'B-0011'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa11.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-012

Objective: Verify the system behavior for product instance 12.

Input Parameters: {serial: 'SN-1012', manufacturer: 'BrandX', batch: 'B-0012'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa12.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-013

Objective: Verify the system behavior for product instance 13.

Input Parameters: {serial: 'SN-1013', manufacturer: 'BrandX', batch: 'B-0013'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa13.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-014

Objective: Verify the system behavior for product instance 14.

Input Parameters: {serial: 'SN-1014', manufacturer: 'BrandX', batch: 'B-0014'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa14.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-015

Objective: Verify the system behavior for product instance 15.

Input Parameters: {serial: 'SN-1015', manufacturer: 'BrandX', batch: 'B-0015'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa15.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-016

Objective: Verify the system behavior for product instance 16.

Input Parameters: {serial: 'SN-1016', manufacturer: 'BrandX', batch: 'B-0016'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa16.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-017

Objective: Verify the system behavior for product instance 17.

Input Parameters: {serial: 'SN-1017', manufacturer: 'BrandX', batch: 'B-0017'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa17.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-018

Objective: Verify the system behavior for product instance 18.

Input Parameters: {serial: 'SN-1018', manufacturer: 'BrandX', batch: 'B-0018'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa18.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-019

Objective: Verify the system behavior for product instance 19.

Input Parameters: {serial: 'SN-1019', manufacturer: 'BrandX', batch: 'B-0019'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa19.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-020

Objective: Verify the system behavior for product instance 20.

Input Parameters: {serial: 'SN-1020', manufacturer: 'BrandX', batch: 'B-0020'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa20.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-021

Objective: Verify the system behavior for product instance 21.

Input Parameters: {serial: 'SN-1021', manufacturer: 'BrandX', batch: 'B-0021'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa21.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-022

Objective: Verify the system behavior for product instance 22.

Input Parameters: {serial: 'SN-1022', manufacturer: 'BrandX', batch: 'B-0022'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa22.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-023

Objective: Verify the system behavior for product instance 23.

Input Parameters: {serial: 'SN-1023', manufacturer: 'BrandX', batch: 'B-0023'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa23.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-024

Objective: Verify the system behavior for product instance 24.

Input Parameters: {serial: 'SN-1024', manufacturer: 'BrandX', batch: 'B-0024'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa24.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-025

Objective: Verify the system behavior for product instance 25.

Input Parameters: {serial: 'SN-1025', manufacturer: 'BrandX', batch: 'B-0025'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa25.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-026

Objective: Verify the system behavior for product instance 26.

Input Parameters: {serial: 'SN-1026', manufacturer: 'BrandX', batch: 'B-0026'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa26.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-027

Objective: Verify the system behavior for product instance 27.

Input Parameters: {serial: 'SN-1027', manufacturer: 'BrandX', batch: 'B-0027'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa27.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-028

Objective: Verify the system behavior for product instance 28.

Input Parameters: {serial: 'SN-1028', manufacturer: 'BrandX', batch: 'B-0028'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa28.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-029

Objective: Verify the system behavior for product instance 29.

Input Parameters: {serial: 'SN-1029', manufacturer: 'BrandX', batch: 'B-0029'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa29.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-030

Objective: Verify the system behavior for product instance 30.

Input Parameters: {serial: 'SN-1030', manufacturer: 'BrandX', batch: 'B-0030'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa30.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-031

Objective: Verify the system behavior for product instance 31.

Input Parameters: {serial: 'SN-1031', manufacturer: 'BrandX', batch: 'B-0031'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa31.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-032

Objective: Verify the system behavior for product instance 32.

Input Parameters: {serial: 'SN-1032', manufacturer: 'BrandX', batch: 'B-0032'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa32.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-033

Objective: Verify the system behavior for product instance 33.

Input Parameters: {serial: 'SN-1033', manufacturer: 'BrandX', batch: 'B-0033'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa33.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-034

Objective: Verify the system behavior for product instance 34.

Input Parameters: {serial: 'SN-1034', manufacturer: 'BrandX', batch: 'B-0034'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa34.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-035

Objective: Verify the system behavior for product instance 35.

Input Parameters: {serial: 'SN-1035', manufacturer: 'BrandX', batch: 'B-0035'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa35.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-036

Objective: Verify the system behavior for product instance 36.

Input Parameters: {serial: 'SN-1036', manufacturer: 'BrandX', batch: 'B-0036'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa36.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-037

Objective: Verify the system behavior for product instance 37.

Input Parameters: {serial: 'SN-1037', manufacturer: 'BrandX', batch: 'B-0037'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa37.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-038

Objective: Verify the system behavior for product instance 38.

Input Parameters: {serial: 'SN-1038', manufacturer: 'BrandX', batch: 'B-0038'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa38.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-039

Objective: Verify the system behavior for product instance 39.

Input Parameters: {serial: 'SN-1039', manufacturer: 'BrandX', batch: 'B-0039'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa39.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-040

Objective: Verify the system behavior for product instance 40.

Input Parameters: {serial: 'SN-1040', manufacturer: 'BrandX', batch: 'B-0040'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa40.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-041

Objective: Verify the system behavior for product instance 41.

Input Parameters: {serial: 'SN-1041', manufacturer: 'BrandX', batch: 'B-0041'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa41.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-042

Objective: Verify the system behavior for product instance 42.

Input Parameters: {serial: 'SN-1042', manufacturer: 'BrandX', batch: 'B-0042'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa42.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-043

Objective: Verify the system behavior for product instance 43.

Input Parameters: {serial: 'SN-1043', manufacturer: 'BrandX', batch: 'B-0043'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa43.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-044

Objective: Verify the system behavior for product instance 44.

Input Parameters: {serial: 'SN-1044', manufacturer: 'BrandX', batch: 'B-0044'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa44.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-045

Objective: Verify the system behavior for product instance 45.

Input Parameters: {serial: 'SN-1045', manufacturer: 'BrandX', batch: 'B-0045'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa45.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-046

Objective: Verify the system behavior for product instance 46.

Input Parameters: {serial: 'SN-1046', manufacturer: 'BrandX', batch: 'B-0046'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa46.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-047

Objective: Verify the system behavior for product instance 47.

Input Parameters: {serial: 'SN-1047', manufacturer: 'BrandX', batch: 'B-0047'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa47.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-048

Objective: Verify the system behavior for product instance 48.

Input Parameters: {serial: 'SN-1048', manufacturer: 'BrandX', batch: 'B-0048'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa48.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-049

Objective: Verify the system behavior for product instance 49.

Input Parameters: {serial: 'SN-1049', manufacturer: 'BrandX', batch: 'B-0049'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa49.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-050

Objective: Verify the system behavior for product instance 50.

Input Parameters: {serial: 'SN-1050', manufacturer: 'BrandX', batch: 'B-0050'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa50.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-051

Objective: Verify the system behavior for product instance 51.

Input Parameters: {serial: 'SN-1051', manufacturer: 'BrandX', batch: 'B-0051'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa51.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-052

Objective: Verify the system behavior for product instance 52.

Input Parameters: {serial: 'SN-1052', manufacturer: 'BrandX', batch: 'B-0052'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa52.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-053

Objective: Verify the system behavior for product instance 53.

Input Parameters: {serial: 'SN-1053', manufacturer: 'BrandX', batch: 'B-0053'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa53.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-054

Objective: Verify the system behavior for product instance 54.

Input Parameters: {serial: 'SN-1054', manufacturer: 'BrandX', batch: 'B-0054'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa54.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-055

Objective: Verify the system behavior for product instance 55.

Input Parameters: {serial: 'SN-1055', manufacturer: 'BrandX', batch: 'B-0055'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa55.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-056

Objective: Verify the system behavior for product instance 56.

Input Parameters: {serial: 'SN-1056', manufacturer: 'BrandX', batch: 'B-0056'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa56.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-057

Objective: Verify the system behavior for product instance 57.

Input Parameters: {serial: 'SN-1057', manufacturer: 'BrandX', batch: 'B-0057'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa57.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-058

Objective: Verify the system behavior for product instance 58.

Input Parameters: {serial: 'SN-1058', manufacturer: 'BrandX', batch: 'B-0058'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa58.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-059

Objective: Verify the system behavior for product instance 59.

Input Parameters: {serial: 'SN-1059', manufacturer: 'BrandX', batch: 'B-0059'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa59.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-060

Objective: Verify the system behavior for product instance 60.

Input Parameters: {serial: 'SN-1060', manufacturer: 'BrandX', batch: 'B-0060'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa60.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-061

Objective: Verify the system behavior for product instance 61.

Input Parameters: {serial: 'SN-1061', manufacturer: 'BrandX', batch: 'B-0061'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa61.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-062

Objective: Verify the system behavior for product instance 62.

Input Parameters: {serial: 'SN-1062', manufacturer: 'BrandX', batch: 'B-0062'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa62.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-063

Objective: Verify the system behavior for product instance 63.

Input Parameters: {serial: 'SN-1063', manufacturer: 'BrandX', batch: 'B-0063'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa63.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-064

Objective: Verify the system behavior for product instance 64.

Input Parameters: {serial: 'SN-1064', manufacturer: 'BrandX', batch: 'B-0064'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa64.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-065

Objective: Verify the system behavior for product instance 65.

Input Parameters: {serial: 'SN-1065', manufacturer: 'BrandX', batch: 'B-0065'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa65.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-066

Objective: Verify the system behavior for product instance 66.

Input Parameters: {serial: 'SN-1066', manufacturer: 'BrandX', batch: 'B-0066'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa66.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-067

Objective: Verify the system behavior for product instance 67.

Input Parameters: {serial: 'SN-1067', manufacturer: 'BrandX', batch: 'B-0067'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa67.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-068

Objective: Verify the system behavior for product instance 68.

Input Parameters: {serial: 'SN-1068', manufacturer: 'BrandX', batch: 'B-0068'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa68.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-069

Objective: Verify the system behavior for product instance 69.

Input Parameters: {serial: 'SN-1069', manufacturer: 'BrandX', batch: 'B-0069'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa69.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-070

Objective: Verify the system behavior for product instance 70.

Input Parameters: {serial: 'SN-1070', manufacturer: 'BrandX', batch: 'B-0070'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa70.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-071

Objective: Verify the system behavior for product instance 71.

Input Parameters: {serial: 'SN-1071', manufacturer: 'BrandX', batch: 'B-0071'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa71.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-072

Objective: Verify the system behavior for product instance 72.

Input Parameters: {serial: 'SN-1072', manufacturer: 'BrandX', batch: 'B-0072'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa72.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-073

Objective: Verify the system behavior for product instance 73.

Input Parameters: {serial: 'SN-1073', manufacturer: 'BrandX', batch: 'B-0073'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa73.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-074

Objective: Verify the system behavior for product instance 74.

Input Parameters: {serial: 'SN-1074', manufacturer: 'BrandX', batch: 'B-0074'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa74.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-075

Objective: Verify the system behavior for product instance 75.

Input Parameters: {serial: 'SN-1075', manufacturer: 'BrandX', batch: 'B-0075'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa75.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-076

Objective: Verify the system behavior for product instance 76.

Input Parameters: {serial: 'SN-1076', manufacturer: 'BrandX', batch: 'B-0076'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa76.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-077

Objective: Verify the system behavior for product instance 77.

Input Parameters: {serial: 'SN-1077', manufacturer: 'BrandX', batch: 'B-0077'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa77.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-078

Objective: Verify the system behavior for product instance 78.

Input Parameters: {serial: 'SN-1078', manufacturer: 'BrandX', batch: 'B-0078'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa78.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-079

Objective: Verify the system behavior for product instance 79.

Input Parameters: {serial: 'SN-1079', manufacturer: 'BrandX', batch: 'B-0079'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa79.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-080

Objective: Verify the system behavior for product instance 80.

Input Parameters: {serial: 'SN-1080', manufacturer: 'BrandX', batch: 'B-0080'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa80.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-081

Objective: Verify the system behavior for product instance 81.

Input Parameters: {serial: 'SN-1081', manufacturer: 'BrandX', batch: 'B-0081'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa81.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-082

Objective: Verify the system behavior for product instance 82.

Input Parameters: {serial: 'SN-1082', manufacturer: 'BrandX', batch: 'B-0082'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa82.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-083

Objective: Verify the system behavior for product instance 83.

Input Parameters: {serial: 'SN-1083', manufacturer: 'BrandX', batch: 'B-0083'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa83.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-084

Objective: Verify the system behavior for product instance 84.

Input Parameters: {serial: 'SN-1084', manufacturer: 'BrandX', batch: 'B-0084'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa84.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-085

Objective: Verify the system behavior for product instance 85.

Input Parameters: {serial: 'SN-1085', manufacturer: 'BrandX', batch: 'B-0085'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa85.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-086

Objective: Verify the system behavior for product instance 86.

Input Parameters: {serial: 'SN-1086', manufacturer: 'BrandX', batch: 'B-0086'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa86.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-087

Objective: Verify the system behavior for product instance 87.

Input Parameters: {serial: 'SN-1087', manufacturer: 'BrandX', batch: 'B-0087'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa87.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-088

Objective: Verify the system behavior for product instance 88.

Input Parameters: {serial: 'SN-1088', manufacturer: 'BrandX', batch: 'B-0088'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa88.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-089

Objective: Verify the system behavior for product instance 89.

Input Parameters: {serial: 'SN-1089', manufacturer: 'BrandX', batch: 'B-0089'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa89.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-090

Objective: Verify the system behavior for product instance 90.

Input Parameters: {serial: 'SN-1090', manufacturer: 'BrandX', batch: 'B-0090'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa90.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-091

Objective: Verify the system behavior for product instance 91.

Input Parameters: {serial: 'SN-1091', manufacturer: 'BrandX', batch: 'B-0091'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa91.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-092

Objective: Verify the system behavior for product instance 92.

Input Parameters: {serial: 'SN-1092', manufacturer: 'BrandX', batch: 'B-0092'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa92.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-093

Objective: Verify the system behavior for product instance 93.

Input Parameters: {serial: 'SN-1093', manufacturer: 'BrandX', batch: 'B-0093'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa93.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-094

Objective: Verify the system behavior for product instance 94.

Input Parameters: {serial: 'SN-1094', manufacturer: 'BrandX', batch: 'B-0094'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa94.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-095

Objective: Verify the system behavior for product instance 95.

Input Parameters: {serial: 'SN-1095', manufacturer: 'BrandX', batch: 'B-0095'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa95.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-096

Objective: Verify the system behavior for product instance 96.

Input Parameters: {serial: 'SN-1096', manufacturer: 'BrandX', batch: 'B-0096'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa96.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-097

Objective: Verify the system behavior for product instance 97.

Input Parameters: {serial: 'SN-1097', manufacturer: 'BrandX', batch: 'B-0097'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa97.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-098

Objective: Verify the system behavior for product instance 98.

Input Parameters: {serial: 'SN-1098', manufacturer: 'BrandX', batch: 'B-0098'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa98.

This test validates the integrity of the data stored on the Ethereum ledger.

Test Case TC-099

Objective: Verify the system behavior for product instance 99.

Input Parameters: {serial: 'SN-1099', manufacturer: 'BrandX', batch: 'B-0099'}

Execution Steps: 1. Register product on blockchain. 2. Verify registration. 3. Transfer to Distributor.

Status: SUCCESS. Blockchain TX: 0xaaaaaaaaaa99.

This test validates the integrity of the data stored on the Ethereum ledger.